SOAP

rigid, msg patterns
standardized
inbult err handle

XML

all

distr enterprise env

REST

if rest dont work, u gotta figure out prob

more response types, smaller msg formats

http

pt-pt

no expensive tools to interact w/ web serv
fast, no expensive Processing

in line with web tech

NoSQL

freer

nesting JSON

avai, distrib

dono schema, evolving schema
Scalability

SQL

desg first, then fill data
data type, row fields

Join

schema, normalized

cant reallocate closely-related
data accross diff servers

Data Integrity
TA - NoSQL only intra-docu
atomicity

REST is the newcomer to the block. It seeks to fix the problems with SOAP and provide a truly simple method of accessing Web services. However, sometimes SOAP is actually easier to use; sometimes REST has problems of its own. Both techniques have issues to consider when deciding which protocol to use.

Before I go any further, it's important to clarify that while both SOAP and REST share similarities over the HTTP protocol, SOAP is a more rigid set of messaging patterns than REST. The rules in SOAP are important because without these rules, you can't achieve any level of standardization. REST as an architecture style does not require processing and is naturally more flexible. Both SOAP and REST rely on well-established rules that everyone has agreed to abide by in the interest of exchanging information.

A Quick Overview of SOAP

SOAP relies exclusively on XML to provide messaging services. Microsoft originally developed SOAP to take the place of older technologies that don't work well on the Internet such as the Distributed Component Object Model (DCOM) and Common Object Request Broker Architecture (CORBA). These technologies fail because they rely on binary messaging; the XML messaging that SOAP employs works better over the Internet.

After an initial release, Microsoft submitted SOAP to the Internet Engineering Task Force (IETF) where it was standardized. SOAP is designed to support expansion, so it has all sorts of other acronyms and abbreviations associated with it, such as WS-Addressing, WS-Policy, WS-Security, WS-Federation, WS-ReliableMessaging, WS-Coordination, WS-AtomicTransaction, and WS-RemotePortlets. In fact, you can find a whole laundry list of these standards on Web Services Standards.

The point is that SOAP is highly extensible, but you only use the pieces you need for a particular task. For example, when using a public Web service that's freely available to everyone, you really don't have much need for WS-Security.

The XML used to make requests and receive responses in SOAP can become extremely complex. In some programming languages, you need to build those requests manually, which becomes problematic because SOAP is intolerant of errors. However, other languages can use shortcuts that SOAP provides; that can help you reduce the effort required to create the request and to parse the response. In fact, when working with .NET languages, you never even see the XML.

Part of the magic is the Web Services Description Language (WSDL). This is another file that's associated with SOAP. It provides a definition of how the Web service works, so that when you create a reference to it, the IDE can completely automate the process. So, the difficulty of using SOAP depends to a large degree on the language you use.

One of the most important SOAP features is built-in error handling. If there's a problem with your request, the response contains error information that you can use to fix the problem. Given that you might not own the Web service, this particular feature is extremely important; otherwise you would be left guessing as to why things didn't work. The error reporting even provides standardized codes so that it's possible to automate some error handling tasks in your code.

An interesting SOAP feature is that you don't necessarily *have* to use it with the HyperText Transfer Protocol (HTTP) transport. There's an actual specification for using SOAP over Simple Mail Transfer Protocol (SMTP) and there isn't any reason you can't use it over other transports. In fact, developers in some languages, such as Python and PHP, are doing just that.

A Quick Overview of REST

Many developers found SOAP cumbersome and hard to use. For example, working with SOAP in JavaScript means writing a ton of code to perform extremely simple tasks because you must create the required XML structure absolutely every time.

REST provides a lighter weight alternative. Instead of using XML to make a request, REST relies on a simple URL in many cases. In some situations you must provide additional information in special ways, but most Web services using REST rely exclusively on obtaining the needed information using the URL approach. REST can use four different HTTP 1.1 verbs (GET, POST, PUT, and DELETE) to perform tasks.

Unlike SOAP, REST doesn't have to use XML to provide the response. You can find REST-based Web services that output the data in Command Separated Value (CSV), JavaScript Object Notation (JSON) and Really Simple Syndication (RSS). The point is that you can obtain the output you need in a form that's easy to parse within the language you need for your application.

As an example of working with REST, you could create a URL for Weather Underground. The API's documentation page shows an example URL of http://api.wunderground.com/api/Your_Key/conditions/q/CA/San_Francisco.json. The information you receive in return is a JSON formatted document containing the weather for San Francisco. You can use your browser to interact with the Web service, which makes it a lot easier to create the right URL and verify the output you need to parse with your application.

Deciding Between SOAP and REST

Before you spend hours fretting over the choice between SOAP and REST, consider that some Web services support one and some the other. Unless you plan to create your own Web service, the decision of which protocol to use may already be made for you. Extremely few Web services, such as Amazon, support both. The focus of your decision often centers on which Web service best meets your needs, rather than which protocol to use.

Soap Vs Rest

SOAP is definitely the heavyweight choice for Web service access. It provides the following advantages when compared to REST:

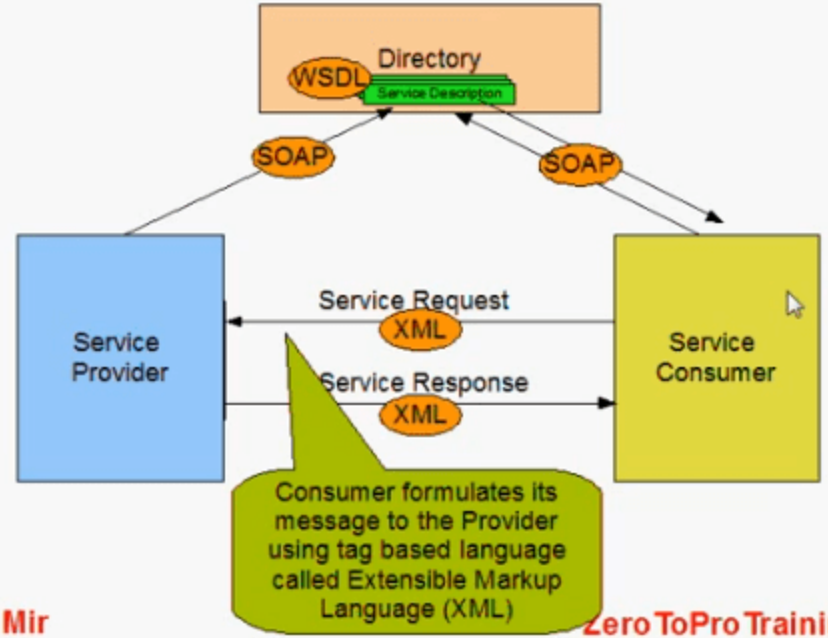
- Language, platform, and transport independent (REST requires use of HTTP)
- Works well in distributed enterprise environments (REST assumes direct point-to-point communication)
- Standardized
- Provides significant pre-build extensibility in the form of the WS* standards
- Built-in error handling
- Automation when used with certain language products

REST is easier to use for the most part and is more flexible. It has the following advantages when compared to SOAP:

- No expensive tools require to interact with the Web service
- Smaller learning curve
- Efficient (SOAP uses XML for all messages, REST can use smaller message formats)
- Fast (no extensive processing required)
- Closer to other Web technologies in design philosophy

XML Message

ZeroToProTraining.com



Introduction to Web Services

HandsonERP

Characteristics

- Resources
  - URI
  - Uniform Interface
    - Methods
    - Representation
- Protocol
  - Client-Server
  - Stateless
  - Cacheable
  - Layered

Those resources are uniquely addressable

4:55 / 13:56

Intro to REST

Google Developers

REST vs. SOAP

REST

- Exposes ***RESOURCES*** which represent ***DATA***
- Uses HTTP Verbs (GET/POST/DELETE)
- Emphasis on simple point-to-point communication over HTTP
- Supports multiple data formats
- Emphasizes stateless communication

SOAP

- Exposes ***OPERATIONS*** which represent ***LOGIC***
- Uses HTTP POST
- Emphasis on loosely coupled distributed messaging
- Supports only XML (and attachments)
- Supports stateless and stateful/conversational operations
- Supports asynchronous messaging
- Strong Typing

ORACLE

7

SOAP vs REST

OracleWebLogic

REST is better than SOAP!

- REST can be consumed by any client, even a web browser with Ajax and Javascript
- REST is lightweight
  - REST doesn't require XML parsing
  - REST consumes less bandwidth – doesn't require a SOAP header for every message
- SOAP is OLD! All the 'cool kids' are using REST!
  - Twitter, Google, Flickr
- I can learn to use REST very quickly
  - It's just nouns and verbs, how hard can it be?
- REST is SAFE!
  - Aren't all 'GET' operations safe?

ORACLE

SOAP vs REST

OracleWebLogic

SOAP is better than REST!

- Building a client for REST can be challenging
  - I can easily generate client-side artifacts from a WSDL
  - I don't want to write raw HTTP calls
  - I don't want to look at the HTTP response code for success/failure – I want to use my own exception types and codes
  - Many IDE's support SOAP development – both client and server
- REST only supports HTTP/HTTPS
  - HTTP is synchronous and in order to scale I need to be able to have asynchronous messaging
- REST is not secure
  - Parameters as part of the URI
  - No support for acquiring tokens
- RESTful services have no contract
  - I have a WADL that specifies URL's but what about schemas for object definition
- REST is not reliable
  - I have to handle failures with retries – no Reliable Messaging
- REST can't be governed
  - How do I know who is consuming my services without a Service Registry?
  - How do I discover RESTful services?

ORACLE

SOAP vs REST

OracleWebLogic

Both SOAP and REST have their rightful places

REST

- Good for:
  - Web Services
  - Limited bandwidth (smaller message size)
  - Limited resources (no xml parsing required)
  - Exposing data over the Internet
  - Combining content from many different sources in a web browser

SOAP

- Good for:
  - Enterprise services
    - High Reliability with WS-RM
    - Transactions with WS-AT
    - Security with WS-Security
  - Asynchronous processing
  - Contract-first development
  - Stateful /conversational operations
  - Standards support, interoperability with business applications
  - Tooling Support

8:12 / 9:51

SOAP vs REST

OracleWebLogic